



INTRODUCTION TO ADVANCED ROBOTICS

PROJECT 8

THE CAPSTONE

WHAT THIS PROJECT BUILDS

Everything from the last seven projects, in one file. The camera reads the board. A function decides the move. The arm picks a token off a supply rack, carries it across, and places it in the right cell. Then it goes back and does it again.

This is the project where the vision stops being a display and starts being an input to a machine that moves.

- **Project 7 gave you a function** — `parse_board_state` — that reads nine cells. It is in this file, nearly unchanged.
- **This project adds a decision** — which cell, and a winner test that has to be fast enough to run on every frame.
- **And an actuator** — a suction pump, because picking up a flat token off a rack needs something that can grip.

● *346 lines. Thirteen TODOs. One file that has to start up, read a board, think, move, and clean up after itself — and it is the last thing you write before the module ends.*

THE LOOP THAT DECIDES EVERYTHING

The main loop is doing more than the previous ones, and it is worth holding all of it in your head before you start typing:

- 1 Read a frame**
Same as Project 7. Nothing new.
- 2 Read the board**
Nine cells into nine characters.
- 3 Check for a winner**
Eight possible lines, tested every frame.
- 4 Draw and show**
The overlay, the status line, the window.
- 5 Wait for a key**
Space makes the robot move. q quits. Nothing happens without a keypress.

Notice what is not on that list: the robot move is not a step in the loop. It happens inside the key handler, and it stops everything else while it runs. That single fact is the shape of this project.

VISION IS ALREADY HERE

This is the part you get to keep. The calibration constants and the whole of `parse_board_state` are copied straight out of Project 7:

```
GRID_SCALE = 0.75      # Scale factor for grid size (1.0 = original)
GRID_OFFSET_X = -10     # Horizontal offset in pixels (+X = right)
GRID_OFFSET_Y = -70     # Vertical offset in pixels (+Y = down)
```

Not one digit changed. If your Project 7 grid was right, this grid is right, and the calibration work you did last week still counts.

- *That reuse is the reason this project is possible. The vision half was already written, tested, and understood — which means you can spend all of this project on the parts that are new: the decision and the arm.*

One difference: the two colour ranges are swapped in meaning. In Project 7 red was the human token. Here the constants say something else. Read them before you trust them.

THE TOKENS CHANGED COLOUR

Same physical game pieces, different assignment. Project 7 used blue for the robot. This project uses red:

```
# Red Tokens = 'O' (Robot Player) - HSV wraps at 0/180
LOWER_RED1 = np.array([0, 120, 100]); UPPER_RED1 = np.array([10, 255, 255])
LOWER_RED2 = np.array([170, 120, 100]); UPPER_RED2 = np.array([180, 255, 255])

# Green Tokens = 'X' (Human Player)
LOWER_GREEN = np.array([40, 80, 80]); UPPER_GREEN = np.array([85, 255, 255])
```

The red ranges are identical to Project 7. The blue ranges are gone, replaced by green. So the hue wraparound you learned last week is still load-bearing — it just now belongs to the robot instead of the human.

- *Nothing in the code tells you which colour is which player. The comments do, and the comments are the only evidence. If you swap the two lines in `parse_board_state` the program will happily play a game where the robot thinks it is you.*

This is worth noticing on purpose. Every constant in the file is a decision somebody made once, and nothing about it is self-evident from the code alone.

EIGHT WAYS TO WIN

A tic-tac-toe board has exactly eight winning lines: three rows, three columns, and two diagonals. That is the whole rule set, and the function that tests for a win is four lines long:

```
def check_winner(b):  
    """Checks for 3-in-a-row winning conditions."""  
    for i in range(3):  
        if b[i][0] == b[i][1] == b[i][2] != ' ': return b[i][0]  
        if b[0][i] == b[1][i] == b[2][i] != ' ': return b[0][i]  
    if b[0][0] == b[1][1] == b[2][2] != ' ': return b[0][0]  
    if b[0][2] == b[1][1] == b[2][0] != ' ': return b[0][2]  
    return None
```

The loop covers rows and columns together — `i` is the index, and each iteration checks one of each. Then two diagonals by hand, because they do not follow the pattern.

Returning the winning character rather than `True` is deliberate. The main loop needs to know `who` won in order to print it on screen, and the string is already sitting there in the board.

READ THE COMPARISON CAREFULLY

This line looks wrong to almost everybody the first time:

```
if b[i][0] == b[i][1] == b[i][2] != ' ': return b[i][0]
```

There is no `and` in it, so how can it possibly mean what it appears to mean? Python chained comparisons compare each element with the next and with the final one. It means exactly:

```
b[i][0] == b[i][1] and b[i][1] == b[i][2] and b[i][2] != ' '
```

- *This is a correct piece of code. Three equal cells, and none of them blank — which is the part the `!= ' '` at the end is doing. Without it, three empty cells would count as a win for nobody, and the game would end on an empty board.*

A chained comparison like this is easier to misread than to write. Nobody is required to use one, but everybody has to be able to read one correctly.

TRY IT, THEN UNDO IT

The AI has one job: find a cell to play. It does it by pretending, checking whether the pretend wins, and putting things back. Four lines of pretending:

```
for player in ['O', 'X']:
    for r in range(3):
        for c in range(3):
            if board[r][c] == ' ':
                board[r][c] = player
                if check_winner(board) == player:
                    return (r, c)
                board[r][c] = ' ' # Undo
```

The outer loop is the important part. It tries `O` first, so the robot checks whether it can win before it checks whether it must block. Getting that order backwards produces a robot that blocks when it should be winning.

- *Those three nested loops, nine cells each, calling a winner test on every free square. Nine tries. Cheap. It runs once per keypress and there is no reason to be cleverer than this.*

THEN CENTRE, THEN ANYTHING

If no single move wins or blocks, the AI stops trying to be clever. Centre first, because it is on more winning lines than any corner, then whatever is left:

```
# 2. Center priority
if board[1][1] == ' ':
    return (1, 1)

# 3. Pick first open square
for r in range(3):
    for c in range(3):
        if board[r][c] == ' ':
            return (r, c)

return None
```

The final `None` is a draw. No cells left, no winner, and the caller turns that into a message and stops.

- *This is a complete opponent in about twenty lines — not minimax, and happy to hand you a win if you play well.*

Choosing the first open square means top-left whenever the centre is taken. Predictable, and occasionally punished.

NINE CELLS, HARD-CODED

The vision half works in pixels. The arm half works in millimetres. Something has to connect them, and it is a dictionary of nine positions measured by hand:

```
CELL_COORDS = {  
    (0, 0): [120.5, -61.5, 138.4, 163.03, 8.84, -61.76], ...  
    (2, 2): [189.0, 46.0, 77.5, 177.55, -3.27, -84.59],  
}
```

The keys are `(row, col)` — the same coordinates the vision produces. The values are six numbers each: X, Y, Z, and three orientation angles.

The Z values are the interesting ones. They are not level: the top row sits at Z 138, 123 and 96, and the bottom row is down near Z 78. The board is not flat, so every cell needs its own height.

- *This is a lookup table, and it is not a shortcut. It is the calibration for a specific arm on a specific table. It cannot be derived — it has to be measured, one cell at a time, with a real pen.*

HOVER, THEN PLACE

Every point in this file that uses `send_coords` is visited twice — once above the thing, once on it. The pattern is so consistent that it is worth memorising as a shape:

```
# 2. Hover over target cell (higher Z, same orientation)
target_hover = [target_x, target_y, 150, target_rx, target_ry, target_rz]
mc.send_coords(target_hover, ARM_SPEED, 0)

# 3. Lower and release token (same orientation)
target_place = [target_x, target_y, target_z, target_rx, target_ry, target_rz]
mc.send_coords(target_place, ARM_SPEED, 0)
```

Hover is a fixed Z of 150. Place is the measured Z from the dictionary. Same X, same Y, same orientation both times — only the height changes, and only on the way down.

- *Moving sideways with a token held over the board is the expensive, slightly dangerous thing a robot arm does. Every "hover" in this file exists to make that move happen at a known, safe height.*

THE PUMP IS ACTIVE LOW

This is the one piece of hardware whose logic is inverted relative to what you would guess. The docstring says it:

```
"""Controls suction pump via GPIO (1=ON, 0=OFF). Active LOW."""
```

Read those three claims together and they appear to contradict each other. Resolve it by trusting the last two words: the pin going LOW turns the pump on.

```
if state == 1:
    GPIO.output(20, 0)
    GPIO.output(21, 0)
else:
    GPIO.output(20, 1)
    GPIO.output(21, 1)
```

- *So asking for the pump to be ON writes a 0, and asking for it to be OFF writes a 1. The function keeps the human meaning ("1 means on") and absorbs the inversion in one place instead of at every call site.*

Two pins, not one. That is a second pump or a second solenoid valve sharing the duty, and both are always driven together.

WHY IT MUST GO HOME

The move sequence ends with a return, and it is not tidiness:

```
# 4. Return home to scan position
print("  Returning home...")
mc.send_coords(POS_HOME_CARTESIAN, ARM_SPEED, 0)
```

The camera can only see the board from the overhead home pose. If the arm stayed over cell (2, 2) after placing a token, the next frame would be of a hand and a token instead of a board.

- *This is a closed loop. The robot never records that it placed a token — it returns to the only viewpoint where it can verify its own work, and believes the camera. Every failure mode that creates is a real one.*

It also explains the three-second sleep on that last move. It is the slowest approach in the file and the one where a collision would put the arm through the board.

TOKENS COME FROM A RACK

A tic-tac-toe game needs up to five O tokens, and they are not lying on the table — they are stacked in a rack the arm reaches into. Each pick goes one token deeper, so the height has to be computed:

```
supply_z = POS_TOKEN_SUPPLY_BASE[2] - (tokens_picked * TOKEN_HEIGHT)
pos_token_supply = POS_TOKEN_SUPPLY_BASE.copy()
pos_token_supply[2] = supply_z
```

Token zero sits at Z 117. Every pick lowers the target by `TOKEN_HEIGHT` of 5mm. After four picks the arm is reaching 20mm further down the rack.

- *The `.copy()` matters. Mutating `POS_TOKEN_SUPPLY_BASE` in place would make the offset cumulative in the wrong direction — each pick would be measured from the last position instead of the rack base, and the arm would walk away from the tokens one pick at a time.*

Five tokens is the most a full tic-tac-toe game needs, because the robot plays O and the human plays X. The rack does not have to be restocked mid-game.

EIGHTEEN SECONDS

Add up the sleeps in the pick-and-place sequence and you get something worth planning around:

Move	Wait
To supply hover	3.0s
Down into the rack	2.5s
Pump on (settle)	1.0s
Back to hover	2.5s
Across to target hover	3.0s
Lower into the cell	1.5s
Pump off (settle)	1.0s
Back to home	3.0s

Seventeen and a half seconds of sleeping, plus the pump's own 0.3s settle each time it is called. Call it **18.1 seconds** — and every one of them is a full stop in the

- *Nothing is drawn and nothing is checked during those eighteen seconds. The window is frozen and the program is deaf.*

WHAT PRESS SPACE MEANS

The robot does not move on a timer or when the board changes. It moves because a human pressed a key. That is the entire interaction model:

```
key = cv2.waitKey(1) & 0xFF

if key == ord(' ') and not winner:
    move = find_best_move(board)
    if move:
        execute_robot_move(move)
```

Four decisions in five lines. Which key was it. Is the game still running. Is there a move at all. Then do the thing. The `and not winner` guard is the one that stops the robot from playing onto a finished board.

Because the move is triggered by a key rather than by a board change, the human is responsible for placing their X before pressing space. Press it on an unchanged board and the robot will play into whatever it

- *This is the most fragile interface in the module and it was chosen for the simplest possible reason: it is easy to explain.*

ORDER OF OPERATIONS AT STARTUP

Before the loop runs, this file does a lot in a specific order, and the order is the whole difficulty:

- 1 Connect, then read back**
get_angles proves the arm is really there.
- 2 Configure the pins**
BCM mode, two output pins, both written OFF.
- 3 Move to home angles**
Then read where that turned out to be in millimetres.
- 4 Open the camera**
After the arm is out of the way.

Step 3 is the subtle one. The arm is told to go to a pose using joint angles, but the rest of the file works in Cartesian millimetres — so it asks where the arm ended up, and that answer becomes the home position every move returns to.

If the arm cannot get there, the handler installs a hard-coded fallback and the program carries on with a position nobody measured.

THE FALLBACK NOBODY MEASURED

Every error handler in this file ends the same way — print, dump a traceback, and substitute a number:

```
except Exception as e:
    print(f"Error moving to home: {e}")
    import traceback
    traceback.print_exc()
    POS_HOME_CARTESIAN = [66, -62, 235, 180, 0, 90] # fallback
```

So if the arm will not move to home, the program does not stop — it remembers a guess and carries on. Every robot move will then end with a return to coordinates that were never checked against the real arm.

- *This is defensible. A capstone that refuses to start is a capstone that teaches nothing. But notice the trade: the fallback converts a loud, obvious failure into a quiet, mechanical one, and the arm will be moved to the wrong place without complaint.*

If you see a traceback during startup, read the fallback line as the explanation for whatever goes wrong later. The two are connected.

BEFORE YOU TYPE ANYTHING

The capstone starter does not run. Copy it across and start it and you get this:

```
File "M2-P8-Starter.py", line 157
    finally:
        ^^^^^^^
SyntaxError: invalid syntax
```

No output at all. Not one line printed, because the file never got as far as running anything.

The reason is in the last few lines: the `finally` block is given to you, correct, but the `try` it belongs to is still an empty TODO, so the two keywords are not attached to anything.

- *Every other project in this module has started with a `NameError` — a runtime error, raised after the file was successfully loaded. This one is a `SyntaxError`, which Python finds before running a single statement. It is a different category of problem, and it is the last thing this module asks you to notice on your own.*

Read from the bottom up for this one. The error points at the last line of the file and the cause is fifty lines above it.

STEP 1 — THE FOUR IMPORTS

Two TODOs at the top of the file. Four imports between them:

```
import cv2
import numpy as np
import time
```

```
import RPi.GPIO as GPIO
```

```
from pymycobot.mycobot280 import MyCobot280
```

- `RPi.GPIO` — the pin interface. This one only exists on a Raspberry Pi, which is why this project cannot be developed on a laptop.
- The other three — identical to every project since Project 1.

- *Note what is absent: no import of the Project 7 functions — this file redefines `parse_board_state` because there is no module to import from.*

STEP 2 — CONNECT AND VERIFY

The capstone has no `try` block around the connection, so this is where a missing arm stops the program. The TODO asks for a read-back, which is what makes it safe:

```
mc = MyCobot280('/dev/ttyAMA0', 1000000)
time.sleep(0.5)

# Verify connection (M1 style - no explicit power_on)
try:
    angles = mc.get_angles()
    print(f"Connected. Current angles: {angles}")
except Exception as e:
    print(f"Warning during init: {e}")
```

The half-second wait is the same one from Project 6 — the board needs time to boot before it will answer. And unlike Project 6 there is no `power_on` call: the comment says this is the Module 1 style, where the arm powers itself.

- *Read the try block carefully. It catches a failure to **read** the arm, not a failure to **create** it. If the constructor raises, this file is already over — there is no handler above it.*

STEP 3 — SET UP THE PINS

Five lines. Three of them are configuration and two are the first thing the pins actually do:

```
GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)
GPIO.setup(20, GPIO.OUT)
GPIO.setup(21, GPIO.OUT)
GPIO.output(20, 1)  # Off initially
GPIO.output(21, 1)
```

Read those last two lines with Step 5 in mind. Writing `1` is what turns the pump **off**, because the pump is active low. The comment is the whole explanation, and it is load-bearing.

- *BCM means "use the pin's broad-com-number", not its physical position on the board. The numbers 20 and 21 are what this file expects; a different wiring would need different numbers here and nowhere else.*

Pins OFF before anything else runs: if the program crashes later, the pump is already in the safe state.

STEP 4 — MOVE HOME, THEN MEASURE

The longest TODO in the file. Everything about the arm's coordinate system is decided here:

```
print(f"Moving to home angles: {POS_HOME_ANGLES}")
try:
    result = mc.send_angles(POS_HOME_ANGLES, ARM_SPEED)
    time.sleep(4.0)
    POS_HOME_CARTESIAN = mc.get_coords()
    print(f"Home position (cartesian): {POS_HOME_CARTESIAN}")
```

Three ideas in five lines. Go to a pose using **joint angles**, wait four seconds for it to get there, then ask **where it ended up in millimetres** and keep that answer.

Two different coordinate systems, because the arm is told where to go one way and is asked where it is another way. The rest of the file is written in the second one.

- *The four-second sleep is not padding. It is the difference between reading the arm's position and reading the position it is still travelling to.*

STEP 5 — THE PUMP FUNCTION

The first function you write in this project, and the shortest:

```
def set_pump(state):  
    """Controls suction pump via GPIO (1=ON, 0=OFF). Active LOW."""  
    if state == 1:  
        GPIO.output(20, 0)  
        GPIO.output(21, 0)  
    else:  
        GPIO.output(20, 1)  
        GPIO.output(21, 1)  
    time.sleep(0.3)
```

The inversion is the whole point. Callers say `set_pump(1)` to turn suction on and the function writes a `0` to the pin. Nothing above this function needs to know that.

- *The trailing sleep is why the move takes 18 seconds instead of 17.5. It is a settle time — the pump needs a moment to build suction before the arm pulls away.*

This function is called four times per move. It is the only piece of shared state in the whole capstone, and it has to be defined before anything that calls it runs.

STEP 6 — THE BOARD READER

You have written this before. It is here, and this is the whole of it:

```
def parse_board_state(frame):  
    """Scans all 9 cells and returns 3x3 matrix."""  
    board = [[' ' for _ in range(3)] for _ in range(3)]  
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
```

One structural difference from Project 7: the colour conversion happens once for the whole frame, before the cells are cut, so each cell is sliced from an already-converted array.

```
m_red = cv2.bitwise_or(cv2.inRange(roi, LOWER_RED1, UPPER_RED1), cv2.inRange(roi, LOWER_RED2, UPPER_RED2))  
m_green = cv2.inRange(roi, LOWER_GREEN, UPPER_GREEN)
```

Two masks, one threshold, two branches — and check the order. Green is tested first, then red, and each branch requires its colour to beat the other as well as beat 800 pixels.

Note the hard-coded 800. Project 7 named it `MIN_PIXELS`; here it is a literal in two places. Same value, same cells, same meaning — but a value you have to change in two spots.

STEP 7 — THE AI

Three priorities, in order. The first is the only interesting one:

```
def find_best_move(board):  
    """Calculates AI move: Checks for win, block, or picks first empty space."""  
    for player in ['O', 'X']:  
        for r in range(3):  
            for c in range(3):  
                if board[r][c] == ' ':  
                    board[r][c] = player  
                    if check_winner(board) == player:  
                        return (r, c)  
                    board[r][c] = ' ' # Undo
```

Try to win as **O** first. Then, in the same loop, try to win as **X** — and since the human winning is the robot blocking, the second pass is the block. Undo every trial that fails.

- *It mutates the board you hand it, and undoes the trial only when the trial fails. On the move it returns, that cell keeps its value. It does not matter today because the board is re-read from the camera on the next frame — but this function is not safe to call twice on one board.*

Nine empty cells, nine winner tests, two passes. Still under a millisecond, and it only runs when someone presses space.

STEP 8 — THE WINNER TEST

Eight lines, one loop, two diagonals. This is the function the AI calls nine times per pass:

```
def check_winner(b):  
    """Checks for 3-in-a-row winning conditions."""  
    for i in range(3):  
        if b[i][0] == b[i][1] == b[i][2] != ' ': return b[i][0]  
        if b[0][i] == b[1][i] == b[2][i] != ' ': return b[0][i]  
    if b[0][0] == b[1][1] == b[2][2] != ' ': return b[0][0]  
    if b[0][2] == b[1][1] == b[2][0] != ' ': return b[0][2]  
    return None
```

This one is called every single frame, not just on a keypress. That is nine cells to parse and eight lines to check, at whatever rate the camera delivers — and it still has time left over.

- *It is also the function that makes the game end honestly. Because it runs every frame, a win is detected the moment the ninth token lands — you never have to press anything to find out you won.*

STEP 9 — PICK AND PLACE

The longest function in the module, and it is almost all `send_coords` and `time.sleep`.

```
def execute_robot_move(target_cell):  
    """Executes pick-and-place sequence to place an 'O' token at target grid cell."""  
    global POS_HOME_CARTESIAN, tokens_picked  
    row, col = target_cell  
    target_coords = CELL_COORDS[(row, col)]  
    target_x, target_y, target_z = target_coords[0], target_coords[1], target_coords[2]
```

```
    supply_z = POS_TOKEN_SUPPLY_BASE[2] - (tokens_picked * TOKEN_HEIGHT)  
    pos_token_supply = POS_TOKEN_SUPPLY_BASE.copy()  
    pos_token_supply[2] = supply_z  
    mc.send_coords(POS_TOKEN_HOVER, ARM_SPEED, 0)
```

Drop into the rack, pick, carry to the cell, lower, release, go home. The `tokens_picked * TOKEN_HEIGHT` part is how the rack lowers five millimetres per token.

- *This is the function that moves hardware. Everything else in the project is arithmetic and image processing. If this one is wrong, something is about to be dropped, or scraped, or both.*

Two globals is not a design choice anyone would make twice. It is readable here because there are exactly two and both are declared in one line.

STEP 10 — OPEN THE CAMERA

Identical to Project 7. This is the point in the file where the vision half and the hardware half meet:

```
cap = cv2.VideoCapture(0)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
```

Read the section header above it: 3. MAIN GAME LOOP, not "calibration". Everything from here down happens while the loop is running, which is why the camera is opened second and not first.

- *Between the calibration and this line sit five function definitions and the arm moving home. The camera is the last thing initialised so that the arm is already out of the way when the first frame is read.*

ALSO IN THE LOOP — WHAT YOU SEE

Given code, but it is the part that makes the output readable. A white box per cell, then a letter inside it:

```
# Draw grid overlay
for r in range(3):
    for c in range(3):
        x1 = GRID_X_MIN + c * CELL_W
        y1 = GRID_Y_MIN + r * CELL_H
        cv2.rectangle(frame, (x1, y1), (x1 + CELL_W, y1 + CELL_H), (255, 255, 255), 2)
```

The same arithmetic as `parse_board_state`, reading the same constants. If the overlay and the reader ever disagree, you have your bug — and they cannot disagree, because both use `GRID_X_MIN` and `CELL_W`.

- *This is the projection step from Project 7. The board a human is looking at and the board being read are the same three rows in the same order — which is what lets a person and a robot agree about a square.*

Colour note: `(0, 255, 0)` is green in BGR and `(0, 0, 255)` is red, so the code draws X in green and O in red, matching the thresholds the reader uses.

STEP 11 — THE GAME LOOP

The last and largest TODO. The parts you have already written:

```
try:
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break

        board = parse_board_state(frame)
        winner = check_winner(board)
```

Two functions per frame. Parse the board, then test it for a winner. The winner is checked on every single iteration, which is why the game notices its own ending without being told.

The `if not ret` guard is doing exactly what it did in Project 7. Two projects from the end of the module and the same three lines are still load-bearing.

STEP 11 — SHOW THE FRAME

The two lines that end the loop body, and they are where the program becomes interactive:

```
cv2.imshow("Mission 02 Capstone - Tic-Tac-Toe AI", frame)
key = cv2.waitKey(1) & 0xFF
```

- `imshow` — draws `frame` in a named window. The name is the one thing you will see on the taskbar.
- `waitKey(1)` — waits one millisecond and returns the key code. The `1` is what makes this a loop instead of a freeze.

These two lines come **after** all the processing, and that order matters. Show, then wait, then go round again. Waiting first would put a stale board on screen for one frame every loop.

- *The one-millisecond wait also stops this loop pinning a CPU core; without it the frame redraws as fast as it can be read and the keyboard never gets a look in.*

STEP 12 — READ THE KEYBOARD

Mask the key code, then compare it. Two separate steps this time, which is the correct shape:

```
key = cv2.waitKey(1) & 0xFF

if key != 255:
    print(f"Key pressed: {key} ({chr(key) if key < 128 else '?'})")
```

The mask happens first and gets its own line, so the comparison below is a plain equality against a number. Compare that with how Project 7 needed parentheses, and you have the whole lesson in two versions.

- *The debug print fires whenever a key arrives. 255 is what `waitKey` returns when nothing was pressed, so that test means "a key was pressed" rather than "a key was q".*

STEP 13 — ACT ON IT

The final TODO. Three branches, and the order of the conditions is the whole design:

```
if key == ord(' ') and not winner:
    print("SPACE pressed - calculating move...")
    move = find_best_move(board)
    if move:
        print(f"Best move: {move}")
        execute_robot_move(move)
    else:
        print("Game is a Draw!")

elif key == ord('q'):
    break
```

The winner guard is on `space`, not the whole loop, so `q` still works after a win. The draw case is handled explicitly rather than doing nothing.

- *One line does all the work: `execute_robot_move(move)` freezes everything for eighteen seconds. The print statements before it are the only record of what the robot decided and why — which is the same argument for evidence that Project 6 made.*

And then the loop goes straight round and re-reads the board, because the robot does not write its own move anywhere. It has to look at it.

THE LAST SEVEN LINES — AND WHY THEY MATTER

Not part of any TODO and never discussed in class, but this block is what makes the program safe to run twice:

```
finally:  
    cap.release()  
    cv2.destroyAllWindows()  
    set_pump(0)  
    GPIO.cleanup()  
    # Don't release servos - keeps arm powered for next run  
    # mc.release_all_servos()
```

- It belongs to the `try` from Step 11, so it runs on a clean exit and on an exception. Both of those, not one of them.
- The order is deliberate. `set_pump(0)` before `GPIO.cleanup()` — the pin interface is only released after the pin is in its safe state.

And the commented-out servo release is a decision, not an oversight. The arm stays powered so the next run does not have to re-home from wherever it was left.

RUN IT

Send it across and start it. On a Pi with the arm and camera attached:

```
scp M2-P8-Starter.py er@192.168.1.149:Documents  
python M2-P8-Starter.py
```

Startup output looks like this — twice, which we will come back to:

```
Connected. Current angles: [0.0, 45.0, -90.0, ...]  
Moving to home angles: [0, 45, -90, -45, 0, 0]  
    send_angles returned: 1  
Home position (cartesian): [66.0, -62.0, 235.0, ...]
```

Then a window opens with the grid overlay and the prompt to press space. Place a green X, press space, and the arm goes to work.

- *Read those startup lines as a claim list. Every one of them is a statement about what the hardware did, printed before the game starts.*

VERIFY THE CLAIMS

The capstone is the largest file in the module, so it makes the most claims. Checked one at a time, without changing any code.

#	The claim	Result
1	Starts up and opens the camera	False
2	Arm returns to the scan pose	True, twice
3	Recognises both token colours	True
4	Finds wins and blocks correctly	True
5	Ends the game on a full board	True

One false out of five, and it is the first one — which means nothing after it ever ran. We will look at the other four too, because they passed in a file that had never executed.

CLAIM 1 — THE FILE THAT NEVER RAN

The program has no top-level `try` around its connection, and no handler around the startup block. So when it failed, it failed immediately and visibly:

```
Traceback (most recent call last):
  File "M2-P8-Base.py", line 83, in <module>
    set_pump(0)
    ^^^^^^^^^
NameError: name 'set_pump' is not defined
```

Line 83 calls it. Line 134 defines it. Python reads a file from the top down, executing each top-level statement as it goes — so when line 83 ran, the `def` statement fifty-one lines below it had not been reached yet.

- *This is a different order of bug from anything else in the module. Not a wrong value, not a missing bracket — a function that did not exist yet at the moment it was called.*

WHAT THE TRACE TEACHES

Two functions in this file are called before they are defined, and only one of them is a bug:

```
board[r][c] = player
if check_winner(board) == player:
    return (r, c)
```

This call is at line 177 and `check_winner` is defined at line 195 — the same problem, eighteen lines apart. And it has never caused a single error.

- **It is inside a function body.** Those lines are not executed when the file loads. They run when `find_best_move` is called, which is long after both functions exist.
- **set_pump(0) was at module level.** Not in a function. It ran during load, in document order, and the name had not been created yet.
- *Same mistake, opposite outcome, decided entirely by indentation. That is the entire lesson.*

THE FIX

The correction is two deleted lines. Here is the block as it appeared before this project was corrected:

```
POS_HOME_CARTESIAN = [66, -62, 235, 180, 0, 90] # fallback
set_pump(0)
time.sleep(1.0)
# ---
```

And here it is in the file you have now — the same block with those two lines gone:

```
POS_HOME_CARTESIAN = [66, -62, 235, 180, 0, 90] # fallback
# ---
```

That is the whole fix. Nothing was moved or renamed: line 287 does the same job, after the camera opens, once it has been defined for 150 lines.

- *And it was redundant even on the timing question. Step 3 already drove both pins OFF during GPIO setup, so the early call was not "too soon" — it was unnecessary.*

Two problems in one line, which is why it survived being written.

CLAIM 2 — TRUE, BUT TWICE

The arm does reach the scan pose. It also does it **twice**, and the startup log shows both:

```
Moving to home angles: [0, 45, -90, -45, 0, 0]
  Sending send_angles command...
  send_angles returned: 1
Home position (cartesian): [66.0, -62.0, 235.0, ...]
Angles after move: [0.0, 45.0, -90.0, ...]
Moving to home angles: [0, 45, -90, -45, 0, 0]
  Sending send_angles command...
```

Five seconds of arm movement, then another five. The entire home sequence appears twice in the file — once inside the Step 4 TODO block near the top, and again as given code after the camera is opened.

- *Worse than slow. The second block re-reads `get_coords` and overwrites `POS_HOME_CARTESIAN` with the same answer. It works by coincidence, not by design, and the ten seconds is the visible symptom of the copy.*

Left in the corrected file on purpose. Inefficient and harmless: find it, choose which copy goes, and be certain before deleting.

CLAIMS 3 TO 5 — AND THESE HELD

Three claims about behaviour, checked directly:

- 1. Both colours are recognised.** Green above 800 pixels and above red reads as X; red above 800 and above green reads as O. The grid constants are identical to Project 7.
- 2. Wins and blocks are correct.** Eight lines tested; the chained comparison requires three equal cells and none of them blank.
- 3. A full board ends the game.** `find_best_move` returns `None`, the caller prints "Game is a Draw!" and moves on.

All three passed. But they passed in a file that had never executed, which is the uncomfortable part: they are correct **by reading**. Reading is how you check code. Running is how you check that the code you read is the code that executes.

● *Four of five claims were true, and the program still had never run.*

THE HABIT

The capstone needed one check, and it was not any of the ones the code invited you to make.

- **Run the file.** Not the function — the file. Call order, import time, and module-level statements only exist at the top level.
- **Ask what runs, and when.** Every top-level statement executes during load, in document order, before any function is called.
- **Indentation is the whole difference.** Two calls precede their definitions. One crashed and one never could.

This is the last bug of the module and the simplest one to catch. Not because it is subtle, but because the check is obvious once you know to do it: start the program. Every piece of hardware in this course has to be switched on before any of it can be trusted.

- *Four hundred lines of capstone code, one missing function, and the whole thing had never been started.*

THE BRIEF

The Capstone

A camera reads nine cells. A twenty-line opponent picks a square. The arm reaches into a rack, picks up a token with a vacuum pump, carries it eighteen seconds across the table, and puts it down.

Then it goes back to the only viewpoint where it can see the board, and checks with its own eyes whether it did what it meant to.

Seven projects of colour thresholds and camera calibration end here: in one function that moves hardware.

YOUR MISSION

Three parts, in order.

- 1. Calibrate the nine cells.** Move the arm to each `CELL_COORDS` entry by hand and correct the values. Record which ones were wrong.
- 2. Time a full move.** Time one pick-and-place and add it up from the code. Then decide whether the game is still playable at that rate.
- 3. Break the loop.** Comment out the return-home at the end of `execute_robot_move`, run one move, and describe exactly what the next frame shows.

Part three is the interesting one. The robot will place a token perfectly well and then be unable to see the board it just played on.

- *A robot that trusts its own eyes is more useful than one that trusts its own memory. That is worth knowing before you build anything that cannot look.*

YOUR DATA LOG

Measurement	Your value	Corrected?
Cell (0,0) Z		
Cell (1,1) Z		
Cell (2,2) Z		
Total move time		
Home pose X/Y/Z		

Record whether each value needed correcting rather than only what you changed it to. A hand-tuned calibration table that has been measured once and never verified is worth much less than one you know the accuracy of.

DEFINITION OF DONE

- ☐ All thirteen TODOs filled in and the file runs on the Pi.
- ☐ No `TODO` markers left anywhere in the file.
- ☐ The robot wins a game it played fairly, and loses one it played badly.
- ☐ The robot blocks a three-in-a-row rather than letting the human take it.
- ☐ Every token lands inside its cell, from the rack, for all five picks.
- ☐ The camera can still read the board after a move.
- ☐ Pulling the power mid-move leaves the pump off when the program exits.

● *The last two are what make it safe to run twice in a row.*

GO FURTHER

- **A real opponent.** Add two-ply lookahead to `find_best_move` and count how many games the current version gives away.
- **Undo the trial properly.** Make `find_best_move` leave the board untouched, then write a loop that proves it.
- **Kill the duplicate.** Remove the second home-move block and time the startup before and after.
- **Take the blocking out.** A queue or a thread so the window stays live during those eighteen seconds.
- **Let the robot start.** Move O to be the first player and give the human X. Then the AI has to open properly.

Pick one. Not all five.

DEBRIEF

Eight projects. One capstone that read a board, chose a move, and moved a real arm to make it.

Everything it used was built earlier — the hue thresholds and the grid from the board scanner, the arm sequences from the gesture game, the habit of writing down what a program claims before trusting it.

And the bug it died of was the oldest one in programming: a function called before it existed. Four hundred lines, four claims correct by reading, and it had never been started.

Run the file.

That is the whole lesson, and it is the one you will need on the day something does not work.